University of Applied Sciences and Arts
Northwestern Switzerland
School of Computer Science
Institute for Mobile and Distributed Systems (IMVS)

Master of Science in Engineering (MSE)

Project P8

Enhancing Web Accessibility Standards

Implementation of User-Centric Customization Interfaces and CSS-based Contrast Optimization

---

Author: **Florian Schnidrig**
Advisor: Prof. Dierk König
Profile: Computer Science
Date: Summer 2026 (TBD)

Contents

1. Abstract

2. Acknowledgement

3. Introduction

The previous project, *Accessibility on the Web – Where we Stand* [1], examined the fundamental principles of digital inclusion within modern web applications. It explored how semantic HTML serves as the backbone for accessibility, provided an overview of both common and specialized native elements, and clarified the roles of ARIA and WCAG. Furthermore, it offered foundational guidance on accessibility debugging, testing, and the practical use of screen readers.

Building upon that foundation, this project addresses essential aspects of accessibility that, while perhaps less technical in a traditional coding sense, are no less vital for an inclusive digital world. While our previous focus remained largely on the structural and technical "under the hood" mechanics, we have yet to explore the visual design landscape. These design choices are far from purely aesthetic; they play a decisive role in determining whether a web application is a welcoming space or a digital dead end for many users.

4. Theory

4.1. CSS Media Queries

CSS Media Queries allow developer to apply different styles based on characteristics of a device or environment displaying a web page. This is often used to create a responsive web page that uses different stylings, based on screen width.

Listing 1 below describes the media query synthax according to W3C.

```
@media <media-query-list> {
  <rule-list>
}
```

Listing 1: Synthax of the media query where `<media-query-list>` contains `<media-type>`s, `<media-feature>`s as well as logical operators.

For web development the `screen` media-type is mainly of interest, but it is also possible to set it to `print` for targeting printers. The default value is `all`, representing both independent options and is suitable for all devices.

Media-feature is where it gets interesting as there are currently over 40 options available to listen and react to. For example, both `max-width` and `max-height`, as well as their opposites `min-width` and `min-height`, are widely used to create responsive web sites with different appearances optimized for different devices. Listing 2 showcases an example using `screen` and `max-width`. Some of these features consider aspects of accessibility and user preferences which make them essential to create an accessible web page considering individual needs of different users, but they are not as widely used yet as they should. The following media-feature values should be considered when building web applications that should include everyone. [2]

```
@media screen and (max-width: 768px) {
  body {
    background-color: lightskyblue;
  }
}
```

Listing 2: An example styling the background color for mobile devices with media-type `screen` and media-feature `max-width`.

4.1.1. Reduced Motion

The `prefers-reduced-motion` media-feature indicated if a user has enabled a setting on their device to minimize the amount of non-essential motion shown to them. If the value is set to `reduced` animations and transitions should be reduced to the bare minimum.

A value set to `no-preference` indicated that the user has no preference on reduced motions, allowing all kinds of animations and movement. [3]

4.1.2. Contrast

With `prefers-contrast` the user can specify if he requests content to be presented with a lower or higher contrast. The value `no-preference` indicates that no setting regarding contrast preference was configured by the user.

If the value is set to `more` a higher level of contrast is requested by the user whilst a value of `less` indicated that the opposite, so a lower contrast, is preferred by the user.

Additionally, `custom` can be set as a value as well, meaning the user prefers a specific set of colors. The color palette is typically specified within the media feature `forced-colors` explained in the following section. [4]

4.1.3. Forced Colors

`forced-colors` detects, if a user agent has a forced colors mode enabled such as Windows High Contrast. This media-feature has either the value `active` if such a mode is enabled or `none` if not.

When activated, this overwrites and restricts colors defined in the style sheet. Color values are not affected by styles defined in CSS but instead forced by the browser at paint time. The colors enforced depend on the context of an element. Additionally, backplates are drawn behind text to ensure legibility to preserve contrast for text placed on top of images.

Developers are not encouraged to use this media query to create a separate design for users with this feature enabled but to make small tweaks to improve usability if a certain part of a page would not work well in the forced colors context such as replacing box-shadow stylings with a border to not lose the contrast of a button that is only elevated from the background with a box-shadow. [5]

4.1.4. Reduced Transparency

When a user has enabled a setting to reduce the transparent or translucent layer effects the `prefers-reduced-transparency` media-feature will be set to `reduced` while a value of `no-preference` indicates default behavior is expected by the user. This can help improve contrast and readability for some users.

This feature is restricted available and not yet fully supported by Firefox and Safari. [6]

4.1.5. Color Scheme

A feature more broadly used compared to the others introduced in this chapter is the `prefers-color-scheme`. It indicates if the user requests a light or dark color theme set through the operating system or a user agent setting. Possible values are `light` for a light color theme and `dark` for a dark color theme. [7]

4.1.6. Inverted Colors

Using inverted colors can have unpleasant side effects such as shadows turning into highlights, reducing the readability of the content. Developers can react to this preference and ensure the pages integrity with this media-feature. This feature is only supported by Safari so far. [8]

4.1.7. Accessing Media Features through JavaScript

The `window` interface provides the `matchMedia()` method, returning a `MediaQueryList` object to check if the `document` matches a media query string as seen below in Listing 3 for the `prefers-color-scheme` query.

```
const isDarkTheme = window.matchMedia("(prefers-color-scheme:
dark)").matches;
```

Listing 3: Checking if the user's system settings prefere a dark color scheme.

This enables developers to not only react to preferences within CSS but also consider the users wishes within business logic and web page specific features. [9]

4.1.8. Changes in the World of Media Queries

Continuous development and improvement take place in the world of CSS media queries but there is currently one big visual accessibility concern that is not considered yet. Different types of colorblindness and other visual impairments in regards of color perception fall short. There is an open issue in the W3Cs csswg-drafts repository proposing an additional `media-feature` called `color-vision-adjustment` that intends to cover that area to enhance web accessibility for users with color vision deficiencies.

This proposal intends to cover various specific types of colorblindness such as `protanopia` (red deficiency), `deuteranopia` (red-green deficiency), `tritanopia` (blue-yellow deficiency) or `achromatopsia` (near total color blindness - grayscale) that were covered in the previous P7 project with bookmarklets simulating these visual impairments.

Having such a `media-feature` would increase the awareness as well as the possibilities to directly react to the needs of users affected which such impairments. [10]

4.1.9. Deprecated Features

There were media types considering accessibility needs like `embossed`, `aural` and `braille`, but they got deprecated, assuming distinctions between device types will become more blurred in the future and due to media-type referring to device categories rather than the media they support. [11], [12]

4.1.10. Security Concerns

Data accessible with media queries can be abused to construct a fingerprint, helping to identify and track a device. To prevent this, browsers may fudge returned values in some manners. Some Browsers also provide the possibility to prevent this abuse in the browser settings, such as “Resist Fingerprinting” in Firefox resulting in many media queries only reporting default values rather than device specific settings. [12]

4.2. CSS Custom Properties

CSS custom properties, also referred to as CSS variables, are entities representing values. These properties can be used throughout the document, evaluating into the same values, depending on the scope it is defined, everywhere they are used. This helps developers to keep an overview and reduces complexity. Such a property is typically set by the custom property syntax, being two dashes `--`, followed by a name, joined with single dashes. To access a custom property the CSS `var()` function must be used. Listing 4 shows an example of custom properties being used to define a `--primary-color` that then can be used on the whole website and is easy exchangeable in one central part to implement different color palates such as a light and dark theme, based on the users preference. [13]

```css
:root {
  --primary-color: #204CCF;
  --primary-color-contrast: #FFFFFF;
}

button.primary {
  background-color: var(--primary-color);
  color: var(--primary-color-contrast);
}

h1, h2, h3 {
  color: var(--primary-color);
}
```

Listing 4: Defining a primary color custom propperty that can be used throughout the website.

It is possible to be more precise when defining a custom property by using the `@property` at-rule, allowing to define the value type with `syntax`, if the property inherits by default with `inherits` and an initial value with `initial-value`. There is a wide range of values possible for `syntax` such as `<color>`, `<number>` and `<url>` to name a few. Listing 5 shows how the `--primary-color` example from before is achieved with this at-rule. [14], [15]

```css
@property --primary-color {
  syntax: "<color>";
  inherits: false;
  initial-value: #204CCF;
}
```

Listing 5: Defining a primary color custom propperty using the `@property` rule to be more precise.

As displayed in Listing 6 below, both variations are also definable with JavaScript using `registerPropperty()` or `setProperty()`. [16], [17]

```javascript
window.CSS.registerProperty({
  name: '--primary-color',
  syntax: '<color>',
  inherits: false,
  initialValue: '#204CCF ',
});

const root = document.documentElement;
root.style.setProperty('--primary-color-contrast', '#FFFFFF');
```

Listing 6: Using both `registerProperty()` and `setProperty()` in JavaScript to create CSS custom properties.

Of course it is not only possible to write, but also read custom properties with the function `getPropertyValue()` shown in Listing 7 to be able to react to changed values and further work with the latest value.

```javascript
const root          = document.documentElement;
const contrastColor = getComputedStyle(root).getPropertyValue('--
contrast-color');
```

Listing 7: With `getPropertyValue()` the previously set CSS custom property can be read in JavaScript again.

4.2.1. Global State Management

The CSS custom property can not only be used to store values such as colors to be applied to properties but also for state management. As the values can be altered and accessed both within CSS, as well as in JavaScript, it lends itself to store configuration data, based on which the user interface adapts.

When a user has not enabled a reduced motion setting in his OS or browser yet deactivates it with some website specific settings, it is possible to store that preference within a custom property, such as `--prefers-reduced-motion: true` and act on that with the `@container` at-rule to apply different styling at runtime similar to Listing 8. [18]

```css
:root {
  --prefers-reduced-motion: false;
}

@container style(--prefers-reduced-motion: true) {
    .animated-content {
        animation: none !important;
    }
}
```

Listing 8: CSS `@container` at-rule used to react to custom properties turning off animations based on user preferences.

Another possibility would be to use the CSS attribute selector to overwrite other custom properties if another color theme is needed. When the users OS and browser settings suggest he wants a light color theme but he configures it differently on the website itself a custom property such as `--prefers-dark-theme: true` could be set to change the applications color palette as shown in Listing 9. [19]

```css
:root {
  --prefers-dark-theme: false;

    --text-color: #222222;
    --bg-color: #FCFCFC;
    --border-color: #EEEEEE;
    --primary-accent: #0056B3;
}

:root[style*="--prefers-dark-theme: true"] {
    --text-color: #EDEDED;
    --bg-color: #212121;
    --border-color: #232323;
    --primary-accent: #6DB3F4;
}
```

Listing 9: CSS attribute selector used to react to custom properties switching to a dark color theme based on user preferences.

4.2.2. Custom Propperty Toggle

In the landscape of modern CSS, developers often face the challenge of managing repetitive declarations, particularly when implementing light and dark themes. Whether using traditional class-based switching or standard `@media (prefers-color-scheme: dark)` blocks, the logic frequently requires duplicating property keys to override their values. A tiny footnote in the CSS specifications allows for a more elegant approach to state management using custom property to toggle values.

According to CSS specifications, a custom property must represent at least one token to be valid. This token can be a simple whitespace, allowing for exploitation to create `boolean` values as shown in Listing 10. [20]

```css
:root {
  --true: ;
  --false: initial;
}
```

Listing 10: Both `' '` and `initial` are valid values for custom properties.

Traditional theming setups require to define variables twice for both the light- as well as the dark-theme. Taking advantage of the pseudo `boolean` values from Listing 10 the toggle logic can be put into a single custom property, demonstrated in Listing 11.

```css
:root {
  --is-light-theme: var(--true);
}

@media (prefers-color-scheme: dark) {
  :root {
    --is-light-theme: var(--false);
  }
}
```

Listing 11: Assigning either `' '` or `initial` debending on the chosen theme allows to create the base for the toggle logic.

Setting the `--is-light-theme` property as prefix when declairing a custom property results either in a valid declaration, if a whitespace is set as prefix, or an invalid declaration, if `initial` is set as the prefix value. Using `var()` with a fallback value is now automatically assigning either the light-theme or dark-theme color, depending on the validity of the light-theme custom properties displayed in Listing 12.

```css
:root {
  --color-text--light: var(--is-light-theme) black;
  --color-text--dark: white;

  --color-background--light: var(--is-light-theme) white;
  --color-background--dark: black;

  --color-text: var(--color-text--light,
                var(--color-text--dark));
  --color-background: var(--color-background--light,
                      var(--color-background--dark));
}
```

Listing 12: Prepending `initial` when setting the value of a custom property results in `var()` taking the fallback argument.

Multiple fallback values, based on different toggle properties, representing different stylings, could be chained with the `var()` CSS function. This allows for a central state management, assigning custom properties used throughout the whole setup in one single place. [21], [22]

4.3. CSS Functions

There are many native CSS functions available that make a developer's life easier. Some of these functions might be useful when it comes to accessibility such as the `contrast-color()` function that became newly available during the implementation of this project in April 2026 and works across the latest devices and browser versions. This function returns either white or black, depending on which of these two colors has a higher contrast to the color passed to the function as parameter. An example with a `button` styling can be seen in Listing 13. [23]

```css
button {
  background-color: var(--button-color);
  color: contrast-color(var(--button-color));
}
```

Listing 13: Setting the text color of a button black or white, depending on the background color

Having either black or white as font color on a background is a very safe approach but might, depending on the background, feel too harsh, making the reading experience less pleasant. Another relatively fresh available CSS function is the `light-dark()`, allowing developers to provide two colors or images. Depending on the active color scheme the first value is returned for the light theme and the second value for the dark theme. In Listing 14 this approach ids shown using custom properties. [24]

```css
body {
  color: light-dark(var(--color-text--light),
                    var(--color-text--dark));
  background-color: light-dark(var(--color-background--light),
                               var(--color-background--dark));
}
```

Listing 14: Defining the text and background color depending on the selected theme with custom properties.

4.3.1. CSS Custom Functions

Whilst the wide range of available functions in CSS covers a lot of use cases there might be scenarios that do not fit these provided possibilities and need combination of functions or even completely different functionalities. This is where the currently still as experimental flagged `@function` at-rule comes into play. This feature has a similar look and feel to the CSS custom properties, also starting with a leading `--` in their name. To declare such a function the `@function` keyword is needed, followed by a custom name and some optional parameters. Both parameters and function name start with `--`, followed by a case-sensitive identifier defined by the user. Finaly a value defined with `result:` is evaluated and the result returned as seen in Listing 15. [25]

```
@function --color-contrast(--color <color>) returns <color> {
    result: oklch(from var(--color) calc((0.5 - l) * infinity) 0 0);
}
```

Listing 15: A custom function that serves the same purpose as `contrast-color()` but was creates before this feature was widely available.

Similar to the CSS custom properties, a CSS data types, such as `<color>`, `<number>`, `<string>` and many more, can be declared for both parameter value types and the return value type on a function.

4.4. Color Spaces

There are many different models to describe colors the human eye can perceive. Each of these models has different strong suits and weaknesses and is used for different applications throughout different areas not only limited to displays on digital devices. For a long time, colors in CSS were defined in the RGB color space and therefore limited to a certain degree. The CSS Colors Module Level 4 specification provided additional support to manipulate colors in CSS in other ways with other color models. Currently there is even a CSS color module level 5 draft in the works at the W3 consortium. [26]

4.4.1. RGB

RGB mixes the primary colors `red`, `green` and `blue` in different saturations. Additionally, to the colors an alpha channel can optionally be added to define the opacity of a color. This way of describing colors has been used in CSS for ages and can be written both with the `rgb()` function or with the hex annotation which would look like `#2d1fb180` to achive the same color as shown in Figure 1. [26]

```
rgb(45 31 177 / 0.5);
```

Figure 1: An example showing how to use `rgb()` with a color from the Kolibri palette with a 50% oppacity

4.4.2. HSL

With the addition of CSS Color Module Level 3 HSL was introduced to CSS where the colors hue is picked based on a color wheel within the RGB color space. The color is described by the `hue` angle on the color wheel, the `saturation` and `lightness` as presented in Figure 2. Additionally, an alpha channel can be added, analogue to RGB. This allows to pick a color and manipulate its saturation or lightness, without having to re-calculate RBG values, allowing the creation of different shades of the same color way easier. [26]

```
hsl(256 82 55 / 0.6);
```

Figure 2: How to use `hsl()` to configure another color from the Kolibri palette with a 60% oppacity

4.4.3. CIELAB Colors

This color space allows to use additional and more vibrant colors outside of the RGB limitation. It is a uniform color space that defines colors based on how they are perceived by the human eye. The `oklab()` function describes colors in that space with `red/green-ness`, `yellow/blueness` along the a and b axis in the Oklab color space. Alternatively `oklch()` uses `lightness`, `chroma` and `hue` to describe colors. These ways of describing colors are consideret the current state of the art and an example can be seen in Figure 3. Both functions accept an optional alpha channel for opacity. [26]

```
oklab(0.638 0.176 -0.279 / 0.7);
oklch(0.718 0.255 301.5 / 0.7);
```

Figure 3: Showcasing `oklab()` and `oklch()` with yet another color from the Kolibri palette with a 70% oppacity

4.4.4. HWB

Independent from the new color space `hwb()`was added, which considers the `hue`, `whiteness` as well as `blackness` within the RGB color space and is similar to `hsl()`. An example is seen in Figure 4. [26]

```
hwb(325 18 0 / 0.3);
```

Figure 4: Using `hwb()` to recreate a different color shade from the Kolibri palette with a 30% oppacity

4.5. Contrast perception

[TODO] Some introducing summary on contrast.

4.5.1. Relative Luminance

The current WCAG 2.x standard bases contrast requirements on relative luminance between text and its background, independent of hue. This assumes that for individuals with color vision deficiencies, hue and saturation have minimal or no effect regarding legibility as assessed by reading performance. Since the inability to distinguish specific colors does not typically impair light-dark perception, color itself is not considered a primary factor in these calculations. Far more important is that smaller and thinner fonts may be rendered by user agents with a much fainter appearance than the color defined in the CSS, leading to a perceived contrast that is notably lower than the theoretical ratio.

The WCAG guideline requires at least a contrast ratio of 3:1 to satisfy the corelating requirement for level AA. This is based in the recommendation from ISO-9241-3. To achieve level AAA certification the minimum ratio is set to 7:1. This ratio was chosen because it compensated for the loss in contrast sensitivity experienced by users with approximately 20/80 vision (This means that someone needs to be 20 feet away to see what a person with normal vision can see from 80 feet away). People with more than this degree of vision loss usually use assistive technologies.

Color deficiencies are so diverse that it is impossible to generalize effective color pairs for contrast based on quantitative data. This is why contrast independent of color perception is so important.

A notable exception is protanopia where red color tones are perceived as dark grey, resulting in a bad contrast on darker colors and black. This is why it is recommended to generally not use red on black. [27]

The formula depicted in Listing 16 is used in the WCAG 2.x specification to calculate the relative luminance used for further calculations such as color contrast. [28]

$$L = 0.2126 \cdot R + 0.7152 \cdot G + 0.0722 \cdot B$$

$$\text{Where} \quad C = \begin{cases} \frac{C_{\text{sRGB}}}{12.92} & \text{if } C_{\text{sRGB}} \leq 0.03928 \\ \left(\frac{C_{\text{sRGB}}+0.055}{1.055}\right)^{2.4} & \text{otherwise} \end{cases}$$

Example: Dodger Blue `rgb(30, 144, 255)`

1. Normalize (/255)

$R_{\text{sRGB}} = \frac{30}{255} = 0.1176$
$G_{\text{sRGB}} = \frac{144}{255} = 0.5647$
$B_{\text{sRGB}} = \frac{255}{255} = 1.0000$

2. Linearize

$R = 0.0123$
$G = 0.2747$
$B = 1.0000$

3. Calculate Luminance (L)

$$L = (0.2126 \cdot 0.0123) + (0.7152 \cdot 0.2747) + (0.0722 \cdot 1.0000) = \mathbf{0.2712}$$

Calculate Contrast Ratio

$$\text{Ratio} = \frac{\text{L1} + 0.05}{\text{L2} + 0.05}$$

Listing 16: Step-by-step calculation of relative luminance based on the W3C sRGB formula. [28]

4.5.2. Perceptual Color Models

The WCAG 2.x contrast guidelines that are based on relative luminance are being replaced in the future with the WCAG 3.0 guidelines that use the Accessible Perceptual Contrast Algorithm (APCA). Massive changes in display technology, web content and CSS functionality led to the approach that was defined nearly 2 decades ago for WCAG 2.x to be outdated and in need for replacement based on advances in vision science since 2005.

The current approach follows a binary nature where the criteria either passes or fails as a whole. It is important to understand the non-linear aspects of perception and using a model that takes these aspects in account.

All perception is context sensitive and when it comes to readability contrast font weight and line thickness are principal factors that must be taken into account when looking at luminance contrast.

The color contrast regarding hue, chroma or saturation is less relevant for readability, but high light/dark contrast ensures the best readability. Smaller and thinner visual characteristics lower the perceived contrast, requiring for the light/darkness difference to increase as showcased in Figure 5.

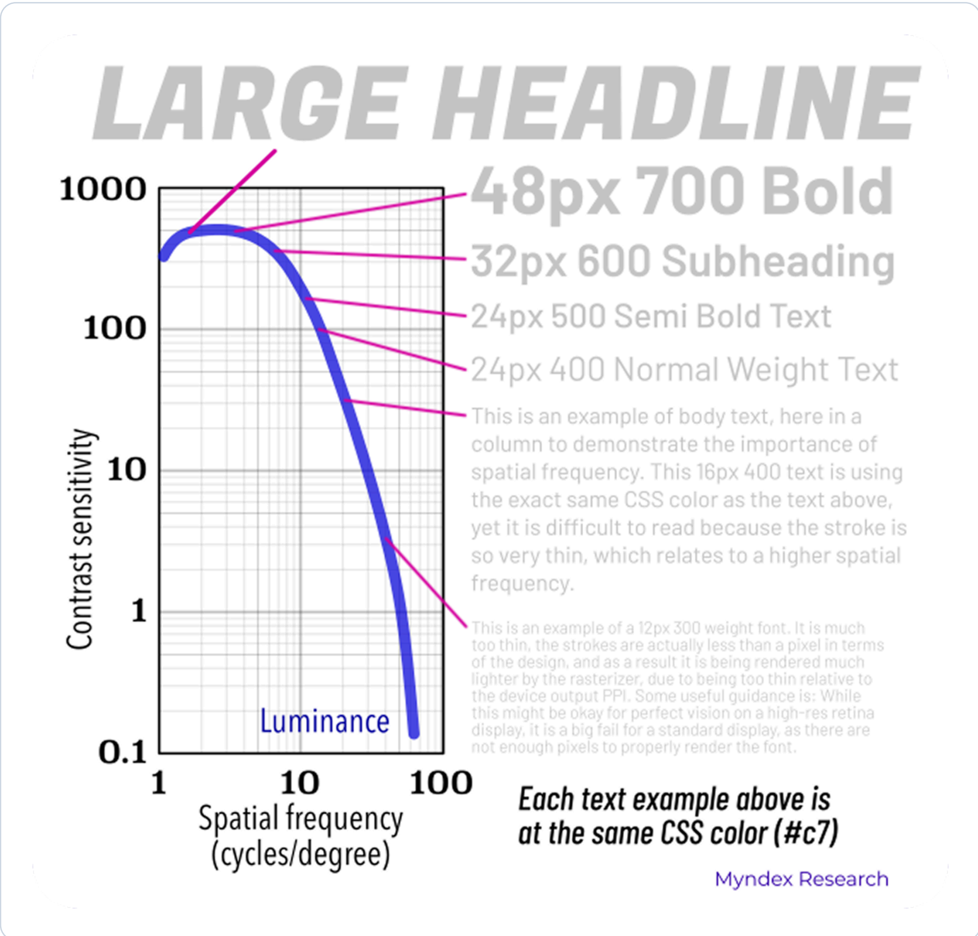

Figure 5: This chart demonstrates with the usage of text samples the spatial dependance of human contrast sensitivity. [29]

Nontextual objects like an icon require a lower lightness contrast compared to text and the contrast requirement of two colors is dependent of the use case, size, thickness and so on. The math behind WCAG 2.x contrast for accessibility has problems that have been known for a long time and are criticized widely. Case studies that compare contrast between white and black text on a colored background found that the variant that was deemed as inaccessible by the guidelines were perceived as more readable by the majority of contestants with color vision deficiencies. [30]

APCA is a new approach for calculating and predicting readability contrast related to color appearance on self-illuminated RGB computer displays introducing the lightness contrast (L^c) value. This new approach considers the context in which two colors are used rather than passing or failing regardless of the use case. [29]

4.6. Canvas

The HTML `<canvas>` element is intended to draw graphics and animations on a website. The element itself does not have a default visual styling and is intended to be manipulated via JavaScript with either the `Canvas API` or the `WebGL API`. With the `Canvas API` it is possible to create game graphics, data visualization, photo manipulations, real-time video processing among other things. `Canvas API` focuses mainly on 2D graphics while `WebGL API` draws hardware-accelerated 2D and 3D graphics. Along manipulation possibilities, it is also possible to extract `ImageData` from a graphic. Listing 17 shows an example how `RGBA` values from an area on a canvas can be extracted in JavaScript. [31], [32], [33]

```
const canvas = document.createElement('canvas');
const ctx    = canvas.getContext('2d');

ctx.fillStyle = 'oklab(0.638 0.176 -0.279)';
ctx.fillRect(0, 0, 100, 100);

const red   = ctx.getImageData(10, 10, 10, 10).data[0];
const green = ctx.getImageData(10, 10, 10, 10).data[1];
const blue  = ctx.getImageData(10, 10, 10, 10).data[2];
const alpha = ctx.getImageData(10, 10, 10, 10).data[3];
```

Listing 17: Creating a purple 100 x 100 pixels square add extracting the RGBA data from a 10 x 10 pixels part at the coordinates x=10 and y=10 from that square.

4.7. Ishihara

The Ishihara color test, named after Japanese ophthalmologist Shinobu Ishihara, was originally designed in 1917 to identify red-green color vision deficiencies. The test utilizes a series of circular plates covered in pseudo-isochromatic dots, randomly sized circles of varying colors that appear identical to those with color blindness but distinct to those with standard vision. [34]

An example of such a plate can be seen in Figure 6 that uses a orange and teal color plate which should be visible independent of the viewers vision types. This color combination was typically used as demonstratin plate at the beginning of a ishihara test.

4.7.1. Functional Application in Contrast Testing

While traditionally a diagnostic tool for the human eye, the mechanics of these plates offer a rigorous framework for testing visual hierarchy and contrast in design.

By packing dots of different colors together, the Ishihara method forces the eye to rely on "chromatic contrast" to find a pattern. In a custom application, if a design's foreground and background colors "bleed" together on a dotted plate, it proves the colors lack the necessary luminance contrast to be accessible.

Custom plates can be generated using specific color palettes to ensure that a brand's primary colors are distinguishable not just to standard viewers, but across the spectrum of for example Protanopia or Deuteranopia.

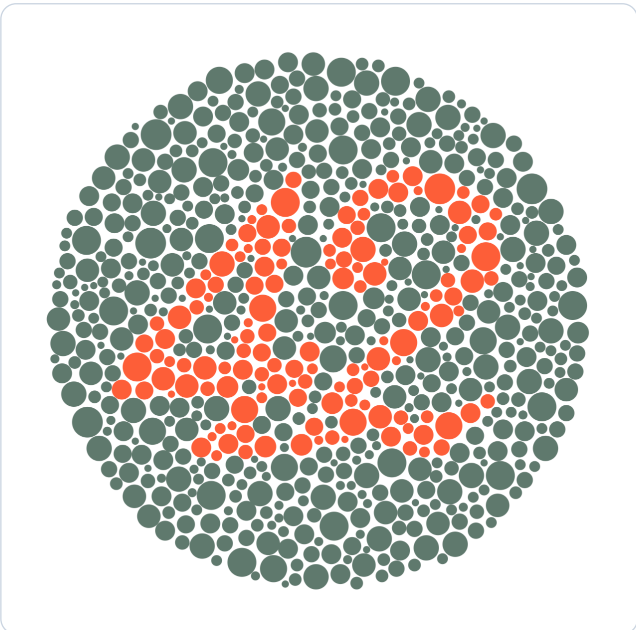

Figure 6: This plate displays a high-contrast figure designed to remain legible across all vision types, serving as the universal baseline for the Ishihara color test.

5. Methodology

6. Implementation

6.1. Preferences Weidget

6.2. Contrast Color Functions

6.3. Interactive Ishihara Plate

7. Discussion

8. Conclusions

Bibliography

[1] Florian Schnidrig, "Accessibility on the Web – Where We Stand." Accessed: Apr. 13, 2026. [Online]. Available: https://accessible-web-initiative.gitbook.io/accessibility-on-the-web-where-we-stand

[2] Mozilla Corporation, "Using media queries." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries/Using

[3] Mozilla Corporation, "prefers-reduced-motion." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-motion

[4] Mozilla Corporation, "prefers-contrast." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-contrast

[5] Mozilla Corporation, "forced-colors." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/forced-colors

[6] Mozilla Corporation, "prefers-reduced-transparency CSS media feature." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-transparency

[7] Mozilla Corporation, "prefers-color-scheme." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme

[8] Mozilla Corporation, "inverted-colors." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme

[9] Mozilla Corporation, "Window: matchMedia() method." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/matchMedia

[10] Naftali Peles, "[mediaqueries] CSS Media Feature for Color Vision Adjustments." Accessed: Mar. 16, 2026. [Online]. Available: https://github.com/w3c/csswg-drafts/issues/10335

[11] World Wide Web Consortium, "Media Queries – disposition of comments." Accessed: Apr. 16, 2026. [Online]. Available: https://www.w3.org/Style/css3-updates/css3-mediaqueries-comments

[12] Mozilla Corporation, "@media." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media#media_types

[13] Mozilla Corporation, "Using CSS custom properties (variables)." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Cascading_variables/Using_custom_properties

[14] Mozilla Corporation, "@property CSS at-rule." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@property

[15] Mozilla Corporation, "syntax CSS at-rule descriptor." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@property/syntax

[16] Mozilla Corporation, "CSS: registerProperty() static method." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/CSS/registerProperty_static

[17] Mozilla Corporation, "CSSStyleDeclaration: setProperty() method." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/CSSStyleDeclaration/setProperty

[18] Mozilla Corporation, "@container CSS at-rule." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/de/docs/Web/CSS/Reference/At-rules/@container

[19] Mozilla Corporation, "Attribute selectors." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Selectors/Attribute_selectors

[20] World Wide Web Consortium, "CSS Custom Properties for Cascading Variables Module Level 1." [Online]. Available: https://www.w3.org/TR/css-variables-1/#syntax

[21] Fynn Ellie Becker, "CSS Custom Property toggles for themes." [Online]. Available: https://fynn.be/blog/css-custom-property-toggles-themes/

[22] Mozilla Corporation, "var() CSS function." Accessed: Apr. 29, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/var

[23] Mozilla Corporation, "contrast-color() CSS function." Accessed: May 04, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/contrast-color

[24] Mozilla Corporation, "light-dark() CSS function." Accessed: May 04, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/light-dark

[25] Mozilla Corporation, “@function CSS at-rule.” Accessed: May 04, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@function

[26] Brian Smith, “New functions, gradients, and hues in CSS colors (Level 4).” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/blog/css-color-module-level-4/

[27] World Wide Web Consortium, “Contrast (Minimum) (Level AA).” Accessed: Apr. 20, 2026. [Online]. Available: https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html

[28] World Wide Web Consortium, “Relative luminance.” Accessed: Apr. 20, 2026. [Online]. Available: https://www.w3.org/WAI/GL/wiki/Relative_luminance

[29] Andrew Somers, “Why APCA as a New Contrast Method?.” [Online]. Available: https://git.apcacontrast.com/documentation/WhyAPCA.html

[30] Ericka O'Connor, “Orange You Accessible? A Mini Case Study on Color Ratio.” [Online]. Available: https://www.bounteous.com/insights/2019/03/22/orange-you-accessible-mini-case-study-color-ratio/

[31] Mozilla Corporation, “<canvas> HTML graphics canvas element.” Accessed: May 11, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/canvas

[32] Mozilla Corporation, “Canvas API.” Accessed: May 11, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

[33] Mozilla Corporation, “ImageData.” Accessed: May 11, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/ImageData

[34] Geoffrey Potjewyd, “The Color Code.” [Online]. Available: https://theophthalmologist.com/issues/2022/articles/sep/the-color-code

List of Figures

List of Aids

Artificial Intelligence and Language Models

Tool: Gemini (Google), Version 3 Flash.

Type of Usage: Used for linguistic refinement of drafts and formating mathematical formulas (Relative Luminance).

Extent: Specific paragraphs regarding WCAG luminance calculations were refined using the tool. All technical facts and code were manually verified for accuracy.